

实验名称: 机器学习实践: 第一次实验报告

实验日期: 2026 年 6 月 28 日

实验地点: C4-519 机房

班级: 人工智能 24-2 班

姓名: 王晨枫

学号: 2024015937

一、实验内容

任务 1: 线性回归

基于 PyTorch 线性回归模型的房价预测实验报告

一、实验题目

基于 PyTorch 的多元线性回归房价预测模型设计与实现

二、实验目的

本实验旨在使用机器学习中的线性回归模型完成房价预测任务。通过读取房价数据集，选取多个房屋相关特征作为输入变量，以房价中位数作为预测目标，使用 PyTorch 框架构建多元线性回归模型，并通过训练集训练模型、测试集评估模型效果，最终实现对房价的预测。

通过本实验，主要掌握以下内容：

- 理解监督学习中输入特征和预测目标的关系；
- 掌握多元线性回归模型的基本原理；
- 学会使用 PyTorch 构建线性回归模型；
- 掌握数据读取、标准化、模型训练、模型测试的基本流程；
- 学会使用 MAE、RMSE、 R^2 等指标评价模型效果；
- 学会分析模型是否存在过拟合或欠拟合问题。

三、实验环境

本实验在 macOS 系统下完成，使用 VS Code 作为代码编辑与运行环境，使用 Miniconda 管理 Python 环境。

3.1 硬件与系统环境

项目	内容
操作系统	macOS
开发工具	Visual Studio Code
环境管理工具	Miniconda / conda
当前 conda 环境	ai
Python 路径	/opt/miniconda3/envs/ai/bin/python
Python 版本	Python 3.10.20
PyTorch 版本	2.11.0

MPS 是否可用 True

3.2 使用的主要库

工具库	作用
PyTorch	构建线性回归模型、计算损失函数、反向传播和参数更新
Pandas	读取和处理 CSV 数据集
NumPy	进行数值计算
Matplotlib	绘制训练损失曲线和预测结果图
scikit-learn	数据标准化、模型评价指标计算

3.3 环境检查过程

在 VS Code 终端中输入以下命令检查当前环境:

```
conda activate ai
which python
python --version
python -c "import sys; print(sys.executable)"
python -c "import torch; print(torch.__version__)"
python -c "import torch; print('MPS 可用:', torch.backends.mps.is_available())"
python -c "import pandas, numpy, matplotlib, sklearn; print('环境正常')"
```

终端输出结果如下:

```
/opt/miniconda3/envs/ai/bin/python
Python 3.10.20
/opt/miniconda3/envs/ai/bin/python
2.11.0
MPS 可用: True
环境正常
```

由此可知, 本实验所需的 Python、PyTorch、Pandas、NumPy、Matplotlib 和 scikit-learn 均已正确安装, 实验环境配置合理。

四、数据集说明

4.1 数据集来源与文件结构

本实验使用的房价数据集已经划分为训练集和测试集, 数据文件存放在项目的“数据集”文件夹中。

项目结构如下:

任务一/

```
|—— .vscode/
|   |—— settings.json
|—— 数据集/
|   |—— train_data.csv
|   |—— test_data.csv
|—— main.py
|—— output/
```

其中:

文件	作用
train_data.csv	训练模型使用的数据
test_data.csv	测试模型预测效果的数据
main.py	完成数据读取、模型训练、测试和可视化的 Python 文件
output/	保存预测结果、图像和模型参数

4.2 数据集字段说明

数据集共有 9 个字段，其中前 8 个字段作为输入特征，最后一个字段作为预测目标。

字段名	含义	是否作为输入
MedInc	区域居民收入中位数	是
HouseAge	房屋年龄	是
AveRooms	平均房间数	是
AveBedrms	平均卧室数	是
Population	区域人口数量	是
AveOccup	平均居住人数	是
Latitude	纬度	是
Longitude	经度	是
MedHouseVal	房价中位数	否，作为预测目标

4.3 数据样例

训练集前几行数据如下:

```
MedInc,HouseAge,AveRooms,AveBedrms,Population,AveOccup,Latitude,Longitude,MedHouse
Val
3.9625,39.0,4.8544,1.0777,998.0,3.2298,34.22,-118.46,1.682
7.3669,16.0,11.2931,2.7280,873.0,1.6724,32.68,-117.17,1.875
2.7768,19.0,4.2804,1.0546,4312.0,2.9413,37.3,-121.85,1.786
```

测试集前几行数据如下:

MedInc,HouseAge,AveRooms,AveBedrms,Population,AveOccup,Longitude,MedHouseVal
3.0658,36.0,4.9435,1.0,310.0,2.5,36.16,-120.37,0.65
1.55,17.0,5.4251,1.0808,672.0,2.0120,37.56,-120.93,1.661
1.1875,29.0,4.2448,1.2378,476.0,3.3287,35.39,-118.98,0.507

五、问题建模

5.1 输入变量与输出变量

本实验的任务是根据房屋相关特征预测房价中位数。

输入变量为:

[x = [x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8]]

其中:

x1 = MedInc
x2 = HouseAge
x3 = AveRooms
x4 = AveBedrms
x5 = Population
x6 = AveOccup
x7 = Longitude
x8 = MedHouseVal

输出变量为:

[y = MedHouseVal]

即房价中位数。

5.2 多元线性回归模型

由于本实验使用 8 个特征预测房价，因此采用多元线性回归模型。模型表达式为:

[=w_1x_1+w_2x_2+w_3x_3+w_4x_4+w_5x_5+w_6x_6+w_7x_7+w_8x_8+b]

其中:

符号	含义
$\hat{y}$	模型预测的房价
(x_1 x_8)	输入特征

(w_1 w_8) 模型需要学习的权重参数

(b) 偏置参数

在 PyTorch 中，该模型可以通过 `nn.Linear(8, 1)` 实现，表示输入维度为 8，输出维度为 1。

六、实验方法与流程

6.1 实验整体流程

6.1 实验整体流程



图 6-1 实验整体流程图

6.2 数据读取

首先使用 Pandas 读取训练集和测试集：

```
train_df = pd.read_csv("数据集/train_data.csv")
test_df = pd.read_csv("数据集/test_data.csv")
```

读取完成后，使用 `head()` 查看数据前几行，确认数据可以正常读取。

6.3 特征选择

本实验选择以下 8 个字段作为输入特征：

```
feature_cols = [  
    "MedInc",  
    "HouseAge",  
    "AveRooms",  
    "AveBedrms",  
    "Population",  
    "AveOccup",  
    "Latitude",  
    "Longitude",  
]
```

选择 `MedHouseVal` 作为预测目标：

```
target_col = "MedHouseVal"
```

6.4 训练集和测试集划分说明

本实验中，数据集已经提前划分为 `train_data.csv` 和 `test_data.csv`。在代码实现中，训练集和测试集分别读取，训练过程只使用训练集，测试集只在模型训练完成后用于评价模型效果。

训练集读取代码：

```
train_df = pd.read_csv(TRAIN_PATH)
```

测试集读取代码：

```
test_df = pd.read_csv(TEST_PATH)
```

模型训练时只使用：

```
X_train_tensor
```

```
y_train_tensor
```

模型测试时才使用：

```
X_test_tensor
```

```
y_test_tensor
```

因此，本实验没有将测试集混入训练过程，训练集和测试集是分开的。

6.5 数据标准化

由于不同特征的数值范围差异较大，例如收入、人口数量、经纬度等字段的量纲不同，如果直接训练，可能导致模型收敛不稳定。因此，本实验使用 `StandardScaler` 对输入特征和目标变量进行标准化处理。

标准化公式为：

$$x' = \frac{x - \mu}{\sigma}$$

其中:

符号 含义

(x) 原始数据

μ 训练集均值

σ 训练集标准差

(x') 标准化后的数据

代码中只使用训练集计算均值和标准差:

```
X_train_scaled = x_scaler.fit_transform(X_train)
```

测试集只使用训练集得到的标准化参数进行转换:

```
X_test_scaled = x_scaler.transform(X_test)
```

这样可以避免测试集信息泄露。

6.6 转换为 PyTorch Tensor

PyTorch 模型不能直接使用 Pandas DataFrame 训练, 因此需要将数据转换为 Tensor:

```
X_train_tensor = torch.tensor(X_train_scaled, dtype=torch.float32).to(device)
```

```
y_train_tensor = torch.tensor(y_train_scaled, dtype=torch.float32).to(device)
```

```
X_test_tensor = torch.tensor(X_test_scaled, dtype=torch.float32).to(device)
```

```
y_test_tensor = torch.tensor(y_test_scaled, dtype=torch.float32).to(device)
```

6.7 构建模型

本实验使用 PyTorch 中的 nn.Linear 构建多元线性回归模型:

```
class LinearRegressionModel(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.linear = nn.Linear(input_dim, 1)

    def forward(self, x):
        return self.linear(x)
```


由于输入特征数量为 8，因此模型输入维度为 8，输出维度为 1。

```
model = LinearRegressionModel(input_dim=8).to(device)
```

6.8 损失函数

本实验使用均方误差作为损失函数：

$$[MSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2]$$

MSE 用于衡量预测值和真实值之间的差距。MSE 越小，说明模型预测结果越接近真实值。

代码如下：

```
criterion = nn.MSELoss()
```

6.9 优化器

本实验使用 Adam 优化器更新模型参数：

```
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

其中，lr 表示学习率，用来控制模型参数每次更新的步长。

6.10 模型训练过程

模型训练过程主要包括以下步骤：

1. 前向传播：模型根据输入特征计算预测房价；
2. 计算损失：使用 **MSELoss** 计算预测值和真实值之间的误差；
3. 清空梯度：防止上一轮梯度影响当前计算；
4. 反向传播：计算损失函数对模型参数的梯度；
5. 参数更新：使用优化器更新模型参数。

训练代码如下：

```
for epoch in range(epochs):
    model.train()

    y_pred = model(X_train_tensor)

    loss = criterion(y_pred, y_train_tensor)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

```
loss_history.append(loss.item())
```

本实验初始参数设置如下：

参数	取值
epochs	2000
optimizer	Adam
learning rate	0.01
loss function	MSELoss
model	nn.Linear(8, 1)

七、实验结果记录

7.1.1 训练过程记录

训练过程中，每隔 100 轮输出一次损失值。部分训练日志如下：

- Epoch [100/2000], Loss: 0.443659
- Epoch [200/2000], Loss: 0.401672
- Epoch [300/2000], Loss: 0.392308
- Epoch [400/2000], Loss: 0.390825
- Epoch [500/2000], Loss: 0.390567
- Epoch [600/2000], Loss: 0.390517
- Epoch [700/2000], Loss: 0.390509
- Epoch [800/2000], Loss: 0.390507
- Epoch [900/2000], Loss: 0.390507
- Epoch [1000/2000], Loss: 0.390507
- Epoch [1100/2000], Loss: 0.390507
- Epoch [1200/2000], Loss: 0.390507
- Epoch [1300/2000], Loss: 0.390507
- Epoch [1400/2000], Loss: 0.390507
- Epoch [1500/2000], Loss: 0.390507
- Epoch [1600/2000], Loss: 0.390507
- Epoch [1700/2000], Loss: 0.390507
- Epoch [1800/2000], Loss: 0.390507
- Epoch [1900/2000], Loss: 0.390507
- Epoch [2000/2000], Loss: 0.390507

从训练日志可以观察到，随着训练轮数增加，Loss 整体呈现下降趋势，说明模型在逐步学习输入特征和房价之间的关系。

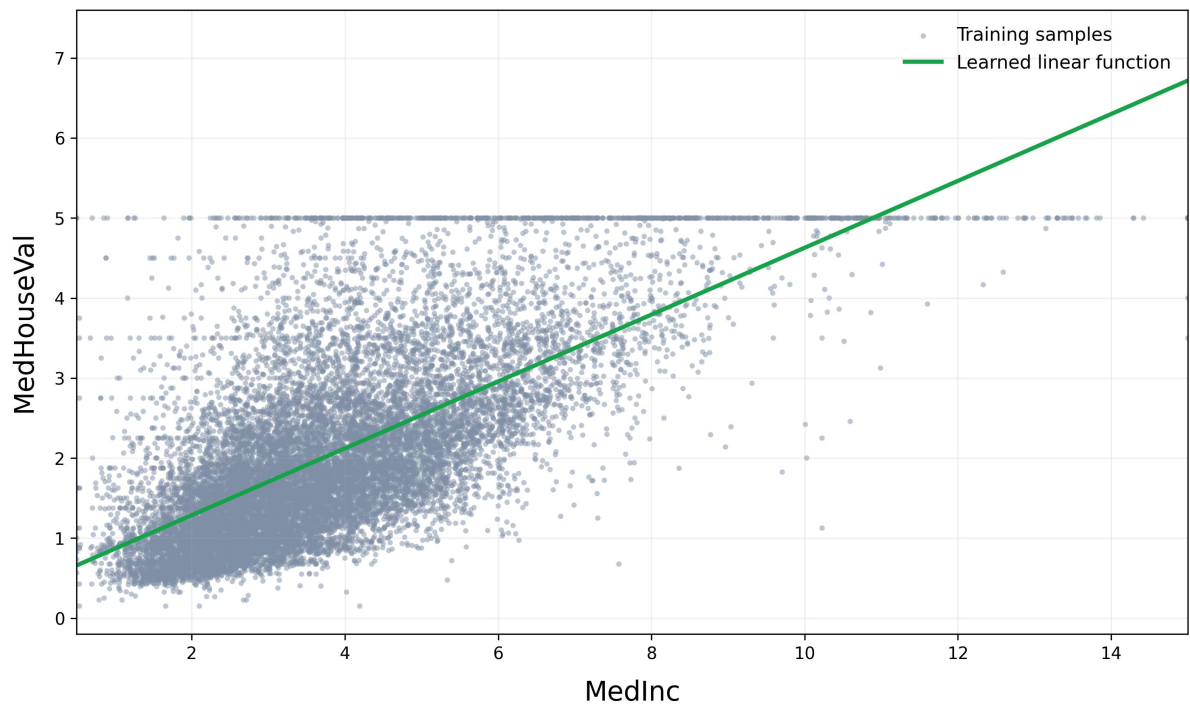
7.1.2 训练集图:



绿色散点：训练集中每个样本的“真实值 vs 预测值”对应点；横轴为真实房价（True Value），纵轴为模型预测房价（Predicted House Value）。红色虚线（Ideal line, $y=x$ ）：理想参考线，代表预测值与真实值完全相等。

整体解读：散点越靠近红色虚线，说明模型对该样本的预测越准确；偏离虚线越远，预测误差就越大。

PyTorch Linear Regression: MedInc -> MedHouseVal



训练样本：训练集中每个样本的真实数据点，横轴是 **MedInc**（家庭收入中位数），纵轴是 **MedHouseVal**（房价中位数）。

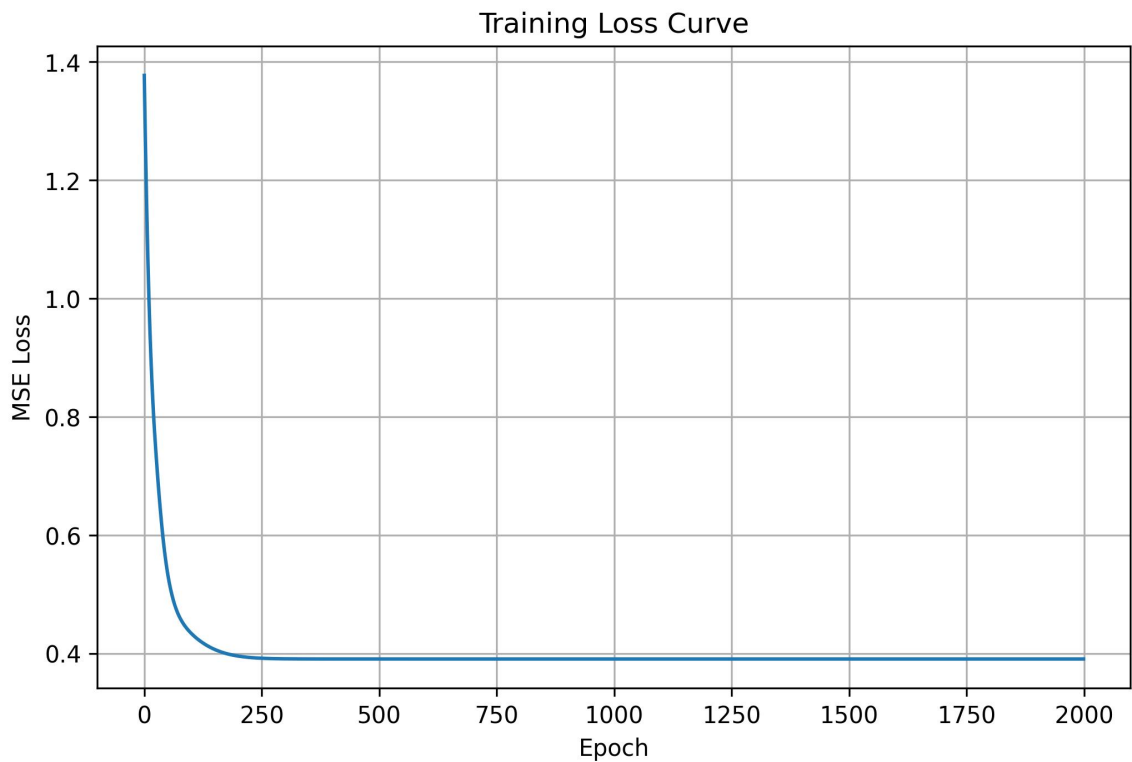
Learned linear function: 模型学到的线性回归函数（拟合直线），表示在单一特征 **MedInc** 下，模型对房价变化趋势的估计。

散点越靠近绿色直线，说明模型在该区间的拟合误差越小；偏离越远，该样本的预测误差就越大

7.2 Loss 曲线

训练完成后，程序保存 **Loss** 曲线图：

output/loss_curve.png



图像说明:

图中横轴表示训练轮数 **Epoch**，纵轴表示均方误差损失 **MSE Loss**。从图中可以看出，随着训练轮数增加，模型损失逐渐下降并趋于稳定，说明模型训练过程正常。

7.3 测试集评价指标

模型在测试集上的评价结果如下:

指标	结果
MAE	0.5423
RMSE	0.7506
R ²	0.5841

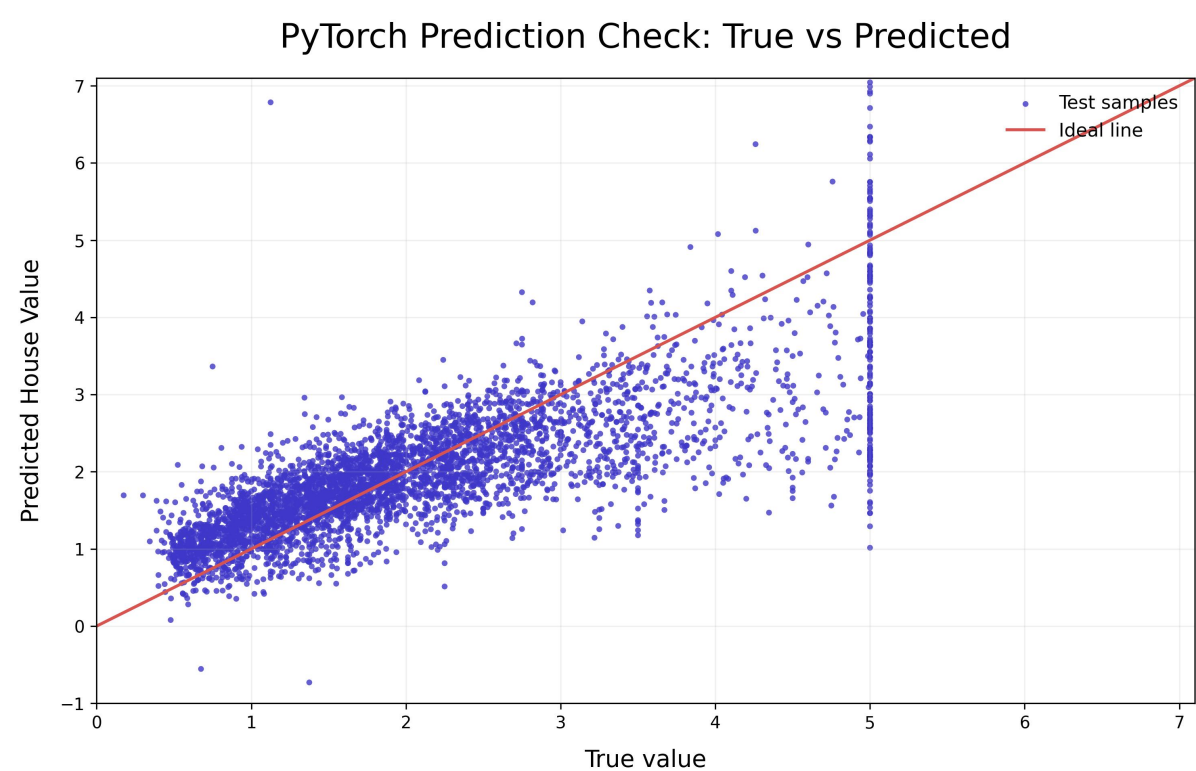
其中:

指标	含义
MAE	平均绝对误差，表示预测值与真实值之间平均相差多少
RMSE	均方根误差，对较大误差更加敏感
R ²	决定系数，表示模型对目标变量变化的解释能力

7.4 真实值与预测值对比图

训练完成后，程序保存测试集真实值与预测值对比图:

output/true_vs_predicted.png



图像说明:

图中横轴表示真实房价，纵轴表示模型预测房价。虚线表示理想情况下预测值等于真实值的位置。如果散点越接近虚线，说明模型预测效果越好。

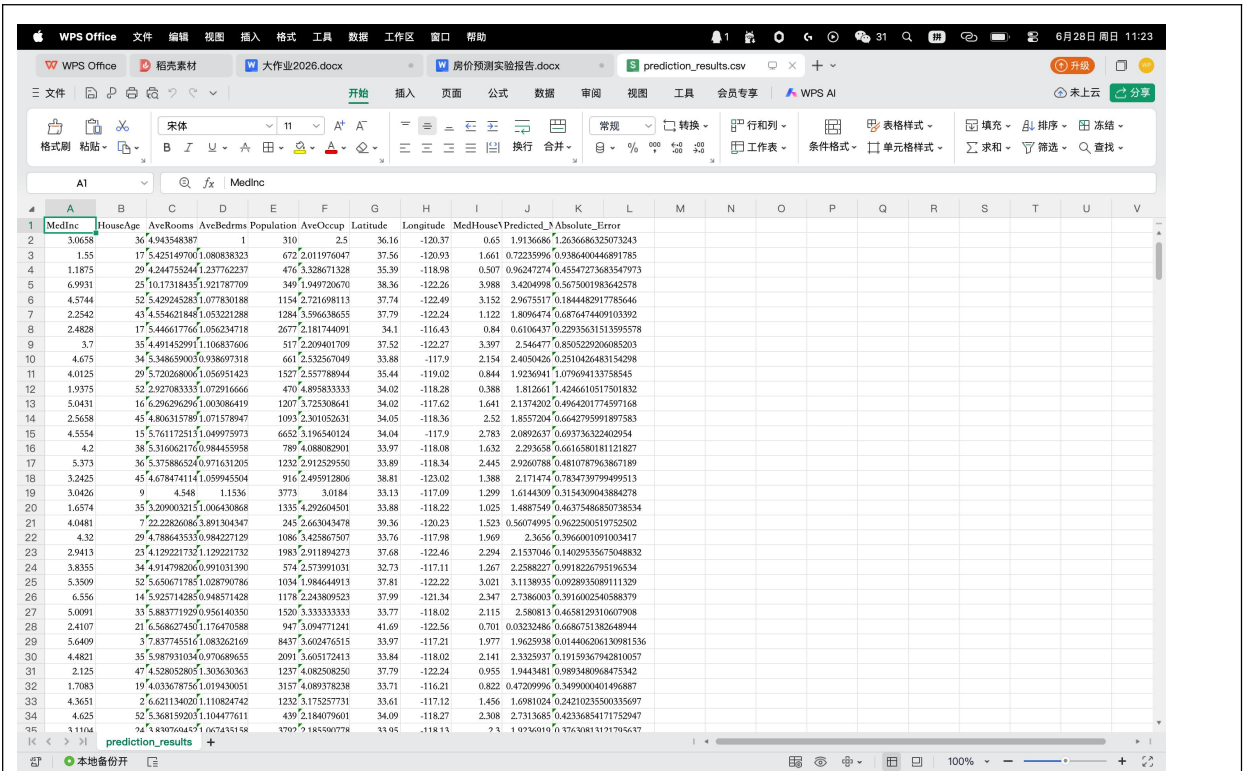
7.5 部分预测结果

模型输出的前 10 个测试样本预测结果如下:

样本编号	真实房价 MedHouseVal	预测房价 Predicted_MedHouseVal	绝对误差
1	0.650	1.913669	1.263669
2	1.661	0.722360	0.938640
3	0.507	0.962473	0.455473
4	3.988	3.420500	0.567500
5	3.152	2.967552	0.184448

完整预测结果已保存至:

output/prediction_results.csv



八、版本迭代设计与调参过程

为突出实验的改进过程，本实验围绕同一数据集设计四个版本。每个版本只改变一个核心方向，并用统一指标进行对比。

版本	核心方案	关键策略	目的
V1	线性回归基线	Linear(8→1), MSELoss, Adam(lr=0.01)	建立可解释基线
V2	线性 + 验证集 + 早停	train/val=8:2 , max_epochs=5000 , patience=80	规范调参流程，减少直接看测试集调参
V3	MLP 扩容模型	8→64→32→1, Dropout , SmoothL1 , AdamW, scheduler	增强非线性表达能力
V4	去除封顶样本后再训练	移除训练集中 MedHouseVal>=5 的 785 条样本	分析封顶标签对主体区间拟合的影响

8.1 V1：线性回归基线

V1 是本实验的最初版本，使用单层线性模型进行房价预测。模型输入为 8 个房屋特征，输出为 1 个房价预测值。该模型结构非常简单，本质上是在学习每个特征对房价的线性影响。

V1 的作用是建立一个基础对照组。后续版本的效果是否提升，都可以与 V1 进行比较。线性回归的优点是训练速度快、结果稳定、模型容易理解；缺点是只能学习线性关系，难以表达房价与收入、地理位置、房龄等因素之间的复杂非线性关系。

插入位置：模型定义与训练配置处

```
model = nn.Linear(len(feature_cols), 1).to(device)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

8.2 V2：加入验证集与早停机制

V2 在 V1 的基础上加入验证集和早停机制。原来的 V1 只使用训练集训练模型，然后直接在测试集上评价。如果不断根据测试集结果调参，就会导致测试集被间接用于模型选择，实验流程不够严谨。

因此，V2 从训练集中再划分出一部分数据作为验证集。训练集用于更新模型参数，验证集用于观察模型是否继续进步，测试集只在最后用于最终评价。这样可以减少数据泄漏风险，使实验流程更加规范。

插入位置：读取 train_df 后、标准化前

```
train_df, val_df = train_test_split(train_df, test_size=0.2, random_state=42)

scaler 只在训练子集 fit, val/test 只 transform
X_train = x_scaler.fit_transform(X_train)
X_val = x_scaler.transform(X_val)
X_test = x_scaler.transform(X_test)
```

插入位置：替换固定 epochs 训练循环

```
best_val_loss, wait = float("inf"), 0
for epoch in range(max_epochs):
    train_one_epoch()
    val_loss = evaluate_on_val()

    if val_loss < best_val_loss - min_delta:
        best_val_loss, wait = val_loss, 0
        best_state = copy.deepcopy(model.state_dict())
    else:
        wait += 1
    if wait >= patience:
        break
model.load_state_dict(best_state)
```

8.3 V3：升级为 MLP 多层感知机

V1 和 V2 使用的都是线性模型，只能学习简单的线性关系。但房价预测问题通常更复杂，房价不仅受收入影响，还可能受地理位置、房龄、房间数量、人口密度等多种因素共同影响，这些因素之间往往存在非线性关系。

因此，V3 将模型从单层线性回归升级为 MLP 多层感知机。模型结构由原来的 $8 \rightarrow 1$ 扩展为 $8 \rightarrow$

64 → 32 → 1。其中，隐藏层可以提取更复杂的特征组合，ReLU 激活函数使模型能够学习非线性关系，Dropout 用于降低过拟合风险。

V3 是本实验中全测试集整体表现最好的版本，说明模型表达能力的提升对房价预测效果有明显帮助。

插入位置：替换 V1/V2 的模型、损失函数与优化器

```
class RegressionMLP(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 64), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(64, 32), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(32, 1)
        )
    def forward(self, x):
        return self.net(x)

model = RegressionMLP(len(feature_cols)).to(device)
criterion = nn.SmoothL1Loss()
optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="min", factor=0.5, patience=20)
```

8.4 V4：去除封顶样本后的对比训练

散点图中 True Value=5.0 附近存在竖直点列，这是房价标签封顶造成的现象。V4 仅从训练集中去除封顶样本，测试集保持不变，以保证评价公平。

插入位置：读取 train_df 后、train/val 划分前；非封顶指标放在测试评估后

```
cap_value = 5.0
cap_count = (train_df["MedHouseVal"] >= cap_value).sum()
train_df = train_df[train_df["MedHouseVal"] < cap_value].copy()
print(f"移除训练集封顶样本数: {cap_count}")

no_cap_mask = test_df["MedHouseVal"].values < cap_value
mae_no_cap = mean_absolute_error(y_true[no_cap_mask], y_pred[no_cap_mask])
```

九、四版本指标对比与迭代分析

以下结果来自统一重跑脚本 output/version_metrics_summary.json。V1—V4 均基于相同 train/test 数据；V2—V4 使用训练集内部 8:2 验证划分。

版本	最佳 val_loss/epoch/stop	Val MAE	Val RMSE	Val R²	Test MAE	Test RMSE	Test R²
V1 线性基线	-	-	-	-	0.5423	0.7506	0.5841
V2 线性+早停	0.4538 / 720 / 800	0.5330	0.7783	0.5303	0.5483	0.9908	0.2753

V3 MLP+早停	0.0994 / 933 / 1013	0.3695	0.5385	0.7751	0.3811	0.5585	0.7697
V4 去封顶 MLP	0.1070 / 1223 / 1303	0.3280	0.4691	0.7673	0.3791	0.5710	0.7594

补充指标: V1 训练集 Train MAE=0.5286, RMSE=0.7195, $R^2=0.6095$; V4 在非封顶测试子集上的 MAE=0.3389, RMSE=0.4925, $R^2=0.7446$ 。

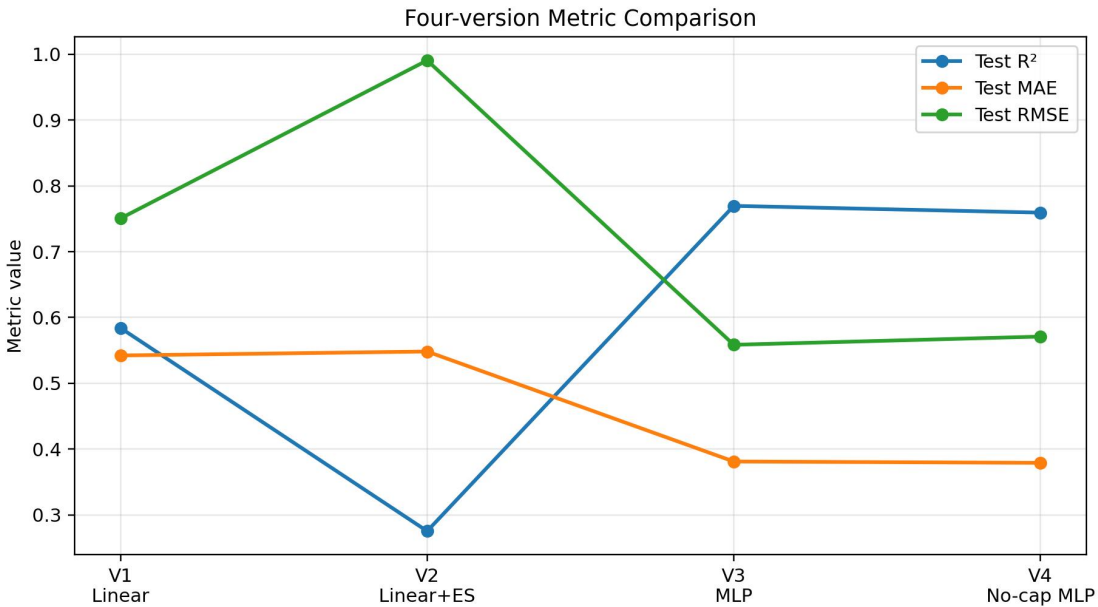


图 5 四版本 Test 指标对比图

9.1 V1 → V2: 流程更规范，但指标下降

V2 加入验证集和早停后，测试 R^2 从 0.5841 降到 0.2753。原因并不是早停机制错误，而是线性模型本身表达能力有限，且训练集被进一步划出验证集后，可用于拟合的数据减少。该结果说明，仅靠训练流程控制不能解决线性模型欠拟合问题。

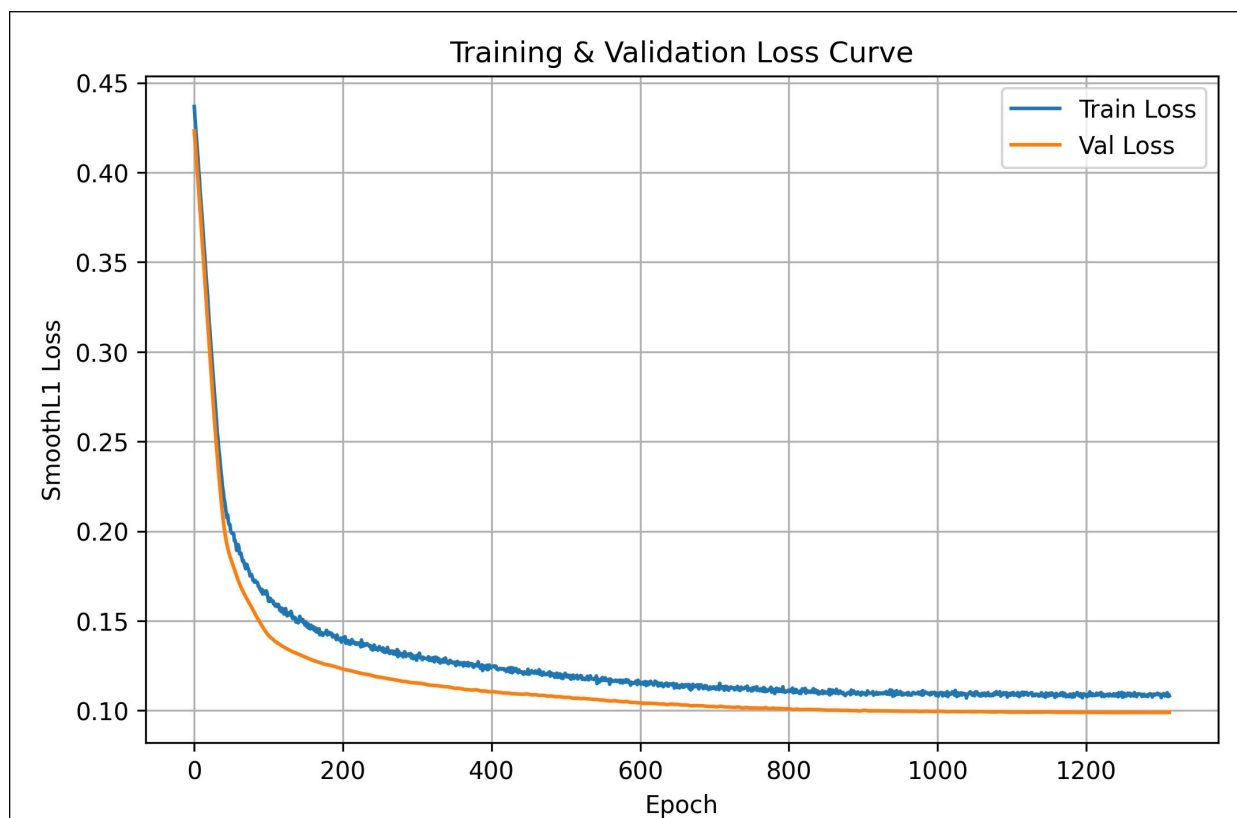
9.2 V2 → V3: 模型扩容带来主要提升

V3 将模型扩展为 MLP，并使用 SmoothL1Loss、AdamW、Dropout 与学习率调度器。Test R^2 从 V2 的 0.2753 提升到 0.7697，RMSE 从 0.9908 降到 0.5585，说明房价预测中存在明显非线性关系，MLP 能学习更复杂的特征组合。

9.3 V3 → V4: 主体区间更稳，但整体略降

V4 去除训练集中 $\text{MedHouseVal} \geq 5$ 的封顶样本后，验证集 MAE 从 0.3695 降到 0.3280，主体价格区间拟合更好；但全测试集 Test R^2 从 0.7697 略降到 0.7594。因此，若目标是全测试集整体效果，V3 更均衡；若关注非封顶主体区间，V4 更有参考价值。

十、可视化结果与封顶样本分析



该图展示了训练集损失 (Train Loss) 和验证集损失 (Val Loss) 随训练轮次 (Epoch) 的变化。两条曲线都呈下降趋势，说明模型在训练过程中不断学习、误差逐渐减小。验证损失低于训练损失，且二者趋势平稳，说明模型在验证集上表现较好，没有明显过拟合。

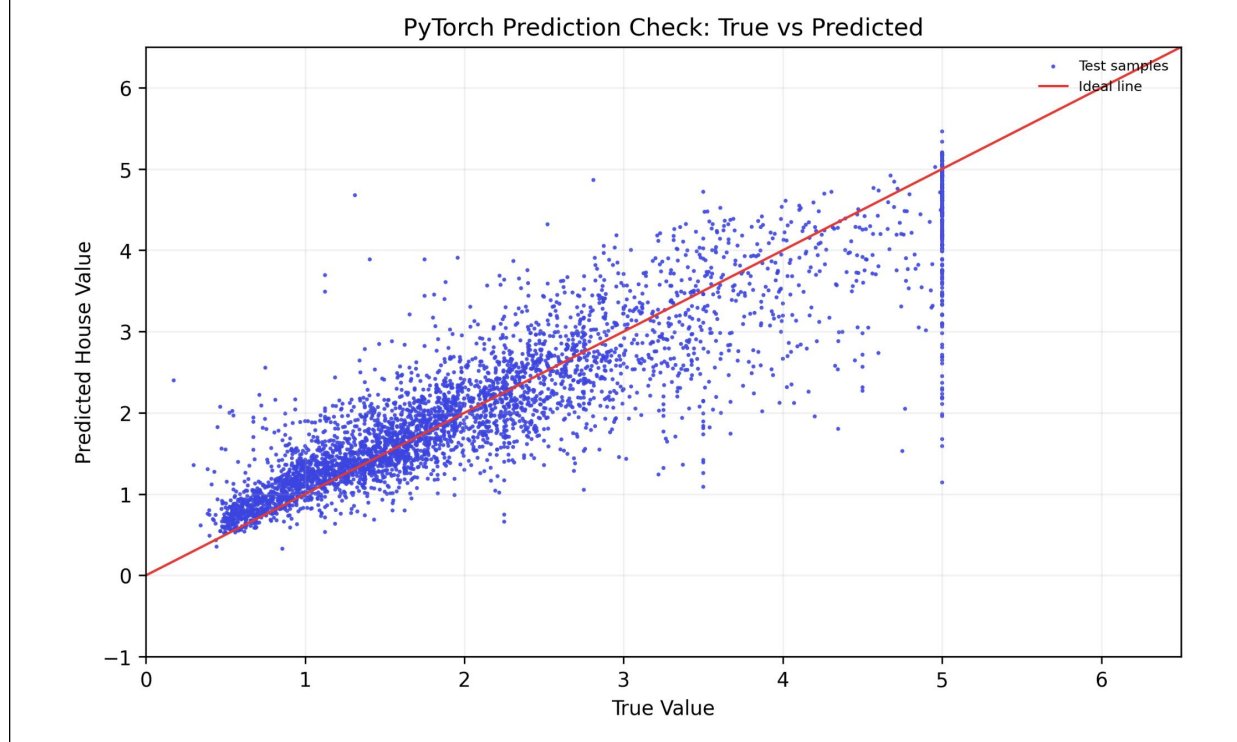


图 6 测试集真实值与预测值对比：右侧竖线反映封顶样本现象

真实值—预测值散点图中，横轴为真实房价，纵轴为预测房价，红线表示理想情况 $y=x$ 。图中右侧 True Value=5.0 附近的竖直点列主要来自标签封顶，而不完全代表模型错误。

多个样本的真实标签都被记录为 5.0 左右，但它们的输入特征不同，因此模型会输出不同预测值。V4 的作用是验证封顶样本是否干扰主体区间拟合，但由于封顶样本仍属于完整数据分布，直接删除并不一定能提高全测试集指标。

十一、过拟合与数据泄漏检查

本实验没有发现明显数据泄漏：测试集始终独立存在，不参与训练和调参；V2—V4 的验证集来自训练集内部划分，只用于早停和调参；StandardScaler 只在训练子集 fit，验证集和测试集只 transform。

检查项	处理方式	结论
训练/测试分离	train_data.csv 与 test_data.csv 分开读取，测试集仅最终评价	无明显泄漏
验证集调参	训练集内部 8:2 划分 train/val	流程更规范
标准化	scaler 只在 train 子集 fit，val/test 只 transform	无标准化泄漏
过拟合控制	Dropout + weight_decay + Early Stopping + scheduler	V3 泛化稳定

从过拟合角度看，V1/V2 参数少，主要问题是欠拟合；V3 参数更多，但引入 Dropout(0.2)、AdamW 的 weight_decay=1e-4、早停和学习率调度器后，验证集 $R^2=0.7751$ 、测试集 $R^2=0.7697$ ，两者接近，说明泛化表现较稳定。

十二、实验输出文件说明

文件	作用
output/train_loss_curve.png / val_loss_curve.png	训练集与验证集损失曲线
output/true_vs_predicted.png	测试集真实值与预测值对比图
output/test_true_vs_pred_no_cap.png	V4 非封顶测试子集预测图
output/prediction_results.csv	测试集逐样本预测结果
output/experiment_comparison.csv	不同实验方案对比结果
output/version_metrics_summary.json	四版本统一重跑指标汇总
output/mlp_regression_model.pth	V3 MLP 最佳模型参数

十三、实验结论

本实验围绕同一房价预测任务完成四个版本的迭代优化。V1 建立线性回归基线，Test R^2 为 0.5841；V2 加入验证集和早停，使流程更严谨，但由于线性模型表达能力不足，测试指标下降；V3 将模型升级为 MLP，并配合 SmoothL1Loss、AdamW、Dropout、Early Stopping 和学习率调度器，Test R^2 提升到 0.7697，成为全测试集整体效果最好的版本；V4 去除封顶样本后，在非封顶测试子集上 MAE 降至 0.3389，但全测试集 R^2 略低于 V3。

综合来看，V3 是本实验最均衡的最终方案。它明显优于线性回归基线，并且验证集与测试集表现接近，说明模型具有较好的泛化能力。V4 适合作为补充分析，用于解释房价封顶样本对可视化分布和主体区间预测的影响。

十四、后续改进方向

- 使用 K-fold 交叉验证，降低单次训练/验证划分带来的随机波动。
- 针对封顶样本尝试样本加权，而不是直接删除，以兼顾主体区间和全分布泛化。
- 引入 LightGBM、XGBoost、随机森林等更适合结构化表格数据的模型作为对照组。
- 按低价、中价、高价区间分别计算 MAE 和 RMSE，分析模型在哪些价格区间误差更大。
- 增加特征重要性或敏感性分析，解释 MedInc、Latitude、Longitude 等特征对预测结果的影响。

所有实验代码与图表数均已上传 Github，可自行查看
<https://github.com/Sunshine-1023/ML-pratice>

成 绩： _____

指导教师（签名）： _____